Name: Mohamed Riaz
Subject: Python for Data Science
Department: Data science & AI
Batch: Jan 2026
Date: Feb 22 2026
Mandatory assignment solution

Question 1:
Consider a list of integers. Write Python code to perform the following operations:

- Accept a list of integers and print each element on a separate line.
- Generate a random list of 15 integers between 1 and 20. Display the list and find the number of unique elements present in it.
- Accept a list of integers and check whether the list is sorted. If the list is not sorted, sort the list in ascending order and display the sorted list.
- Accept a list of integers and split the list into two separate lists:
  - one containing even numbers
  - one containing odd numbers
  - Display both lists.
- Accept a list of integers and determine the frequency of each element. Identify the element with the least frequency and display the count of that element. Assume only one element has the minimum frequency.

In [2]:
```
# ------------------------------------------------------------------------
# 1.a Accept a list of integers and print each element on a separate line.
# ------------------------------------------------------------------------
inputList = input("Enter a list of integer with to comma separate them")
# Cleanup of spacing to avoid issues
inputList = inputList.replace(" ","")
# Get a list of string list
intListStr = inputList.split(",")
# typecast them into int
intList = [int(i) for i in intListStr]
# now print in each line
for n in intList:
    print(n)
```

```
1
6
3
5
2
4
1
3
2
8
5
4
6
```

In [3]:
```
# ------------------------------------------------------------------------
# 1.b Generate a random list of 15 integers between 1 and 20.
#       Display the list and find the number of unique elements present in i
# ------------------------------------------------------------------------

import random
random.seed(0) # for consistant values

randList = [random.randint(1,20) for i in range(0,15)]
print("Random list: ", randList)
```

```
# now find unique numbers
print(set(randList))
```

```
Random list:  [13, 14, 2, 9, 17, 16, 13, 10, 16, 12, 19, 7, 17, 5, 10]
{2, 5, 7, 9, 10, 12, 13, 14, 16, 17, 19}
```

In [4]:
```
# ------------------------------------------------------------------------
# 1.c Accept a list of integers and check whether the list is sorted.
#       If the list is not sorted, sort the list in ascending order and disp
# ------------------------------------------------------------------------
intList = []
n = 5
while n>0:
    intList.append(int(input("enter a number")))
    n=n-1

print("list of integers: ", intList)

sortedList = sorted(intList)

# check if user provided list is already sorted:
if intList == sorted:
    print(intList)
else:
    print(sortedList)
```

```
list of integers:  [1, 5, 2, 4, 3]
[1, 2, 3, 4, 5]
list of integers:  [4, 3, 7, 2, 1]
[1, 2, 3, 4, 7]
```

In [5]:
```
# ------------------------------------------------------------------------
# 1.d Accept a list of integers and split the list into two separate lists:
#       - one containing even numbers
#       - one containing odd numbers
#       - Display both lists.
# ------------------------------------------------------------------------

myList = [int(input()) for i in range(0,6)]

print("myList: ", myList)

odd = []
even = []
for i in myList:
    if i%2 == 0:
        even.append(i)
    else:
        odd.append(i)

print("even: ", even)
print("odd: ", odd)
```

```
myList:  [6, 3, 7, 2, 3, 1]
even:  [6, 2]
odd:  [3, 7, 3, 1]
myList:  [5, 4, 8, 7, 2, 1]
even:  [4, 8, 2]
odd:  [5, 7, 1]
```

In [6]:
```
# ------------------------------------------------------------------------
# 1.e Accept a list of integers and determine the frequency of each element.
#       Identify the element with the least frequency and display the count
#       Assume only one element has the minimum frequency.
# ------------------------------------------------------------------------

a = [1,2,1,3,3,4,5,4,4,5]
d = {}
for i in set(a):
    count = a.count(i)
    d[i] = count
    print("Count of ", i, " is ", count)
```

```
Count of  1  is  2
Count of  2  is  1
Count of  3  is  2
Count of  4  is  3
Count of  5  is  2
```

Question 2:
You are designing an attendance system. In which attendance of the day is maintained in the given format.
attendance_records = [("A","P"), ("B","A"), ("C","P"), ("D","A"), ("E","P"), ("F","P"), ("G","A"), ("H","P"), ("I","P"), ("J","P")]

- Calculate the total no of present count and absence count of the day.
- t seems attendance of the student is wrong, check whether the attendance is actually wrong, if so change it in place. (take the name of the student and new statues as input.)
- Suppose you are adding a new student to the list. Add using list methods.

In [7]:
```python
attendance_records = [("A","P"), ("B","A"), ("C","P"), ("D","A"), ("E","P"),

# helper function
def IsPresent(val):
    if val == "P":
        return True
    else:
        return False

def listToMap(record: list[tuple[str, str]]) -> dict[str,str]:
    d = {}
    for i in record:
        if len(i) != 2:
            print("invalid record")
            return {}
        d[(i[0])] = i[1]
    # print("dict: ", d)
    return d


# func to calculate total number of present and absent count
def TakeAttendance(record):
    present, absent = 0, 0
    if len(record) == 0:
        print("Invalid attendance record")
        return

    for i in attendance_records:
        if len(i) != 2:
            print("Invalid record")
            return
        student = i[0]
        status = i[1]
        if IsPresent(status):
            present += 1
        else:
            absent += 1

    print("present count: ",present)
    print("absent count: ", absent)

TakeAttendance(attendance_records)


# Attendance correction
def CorrectAttendance(name, newStatus):
    d = listToMap(attendance_records)
    if name not in d:
        print("invalid student name")
    d[name] = newStatus
```

```
    newAttendanceRecord = [ (key, val) for key,val in d.items()]
    print("updated attendance record: ", newAttendanceRecord)
    return newAttendanceRecord

print("Before correction: ",attendance_records)
attendance_records = CorrectAttendance("G", "P")

# Adding new student
def AddStudent(name, status):
    attendance_records.append((name, status))

AddStudent("Z", "A")
print("post new student: ", attendance_records)
```

```
present count:  7
absent count:  3
Before correction:  [('A', 'P'), ('B', 'A'), ('C', 'P'), ('D', 'A'), ('E',
'P'), ('F', 'P'), ('G', 'A'), ('H', 'P'), ('I', 'P'), ('J', 'P')]
updated attendance record:  [('A', 'P'), ('B', 'A'), ('C', 'P'), ('D', 'A'),
('E', 'P'), ('F', 'P'), ('G', 'P'), ('H', 'P'), ('I', 'P'), ('J', 'P')]
post new student:  [('A', 'P'), ('B', 'A'), ('C', 'P'), ('D', 'A'), ('E',
'P'), ('F', 'P'), ('G', 'P'), ('H', 'P'), ('I', 'P'), ('J', 'P'), ('Z',
'A')]
```

Question 3:

A school has two events, and the attendees are stored in two sets. Create 2 sets with names of students. Write a program to:

- Find the students who attended both events.
- Find the students who attended only one of the events.
- Find all students who attended at least one event. (use '|' operator)

In [8]:
```
event_a = {"ra", "da", "sa", "pa", "ma", "ga"}
event_b = {"ra", "ca", "ea", "sa", "ma", "la" }

# Lets see what methods we have first
print(set.__doc__)

# we will have to intersection to find common ones
attended_both = event_a.intersection(event_b)
print("Attended both event: ", attended_both)

# now only one event
attended_one = event_b.symmetric_difference(event_a)
print("Attended only one: ", attended_one)

# Find all who attended atleast 1 event
print("Atleast one: ", event_a | event_b)
```

```
Build an unordered collection of unique elements.
Attended both event:  {'ra', 'sa', 'ma'}
Attended only one:  {'la', 'da', 'pa', 'ca', 'ea', 'ga'}
Atleast one:  {'ca', 'sa', 'la', 'ea', 'da', 'ra', 'ga', 'pa', 'ma'}
```

Question 4:

You are given a dictionary containing student names as keys and their marks as values.

```
students = {
"Rohan": 85,
"Spoorthi": 90,
"Aditi": 78,
"Tanya": 92 }
```

Write a program to:

- Find the student with the highest marks.
- Calculate the average marks for the class.

- Add a new student and their marks to the dictionary.

In [9]:
```
students = { "Rohan": 85, "Spoorthi": 90, "Aditi": 78, "Tanya": 92}

print("student with highest score: ", max(students, key=lambda k: students[k

n = len(students)
total = sum(students.values())
print("average marks ", total/n)

# add new student
students["Riaz"] = 99
print("new students: \n",students)
```

```
student with highest score:  Tanya
average marks  86.25
new students:
 {'Rohan': 85, 'Spoorthi': 90, 'Aditi': 78, 'Tanya': 92, 'Riaz': 99}
```

Question 5:

Given a sentence, write a program to:

- Count the number of vowels and consonants.
- Find the longest word in the sentence.
- Reverse the sentence.

In [10]:
```
s = "Hi my name is Riaz, and i am full stack developer"
vowels = ["a", "e", "i", "o", "u"]
vCount = 0
s = s.strip(" ")
for i in vowels:
    vCount+=s.count(i)
cCount = len(s) - vCount
print("vowels count: ", vCount )
print("consonants count: ", cCount )

s = "Hi my name is Riaz, and i am full stack developer"
words = s.split(" ")
print(words)
print("longest word: ", max(words, key=len))

print("reverse: ", s[::-1])
```

```
vowels count:  15
consonants count:  34
['Hi', 'my', 'name', 'is', 'Riaz,', 'and', 'i', 'am', 'full', 'stack', 'deve
loper']
longest word:  developer
reverse:  repoleved kcats lluf ma i dna ,zaiR si eman ym iH
```

Question 6:

A restaurant chain wants to calculate the final cost of a meal, taking into account various discounts, service charges, and taxes. Write a program that has 3 functions that call each other to get the final price.

Each step involves multiple conditions based on default parameters:

- `apply_discount` : This function reduces the base price by a default discount rate (e.g., 10%). Additionally, if the customer is a member, it applies an additional loyalty discount (default 5%). If it's a special promotion day, it applies a further discount (default 3%).
- `add_service_charge` : Calls apply_discount, then adds a service charge based on the type of meal (dine-in or takeout) with default rates. It adds a flat surcharge for dine-in orders, whereas for takeout, it calculates a percentage of the discounted price.

- `calculate_final__price` : Calls add service_charge and applies a tax. For high-value orders (default threshold $100), an additional luxury tax (default 2%) is applied. In the main program, call calculate final__price with parameters to test different scenarios. Make sure to use default parameters wherever required.

In [11]:
```
def apply_discount(price: float, isMember: bool=False, hasSpecialPromotion:
    if price <= 0:
        return 0
    if isMember:
        # apply member discount: 5%
        price = price * 0.05
    if hasSpecialPromotion:
        # apply promotion discount: 3%
        price = price * 0.03
    # apply default discount: 10%
    price = price * 0.1
    return price


def add_service_charge(price: float, isTakeout:bool, isMember: bool=False, h
    price = apply_discount(price,isMember, hasSpecialPromotion)

    if isTakeout:
        price = price
    else:
        price = price + dineIn_surcharge

# Default params
high_val_threshold = 100
default_tax = 0.05
dineIn_surcharge = 20

def calculate_final__price(price: float, isTakeout:bool, isMember: bool=Fals
    add_service_charge(price, isTakeout, isMember, hasSpecialPromotion)

    # add luxury tax, if beyond threshold
    if price > high_val_threshold:
        price = price + price*0.02

    # add general tax
    price = price + price * default_tax

    return price


orders = [
    (200, False, False, True),
    (140, True, False, True),
    (120.65, True, True, False),
    (223.4, False, True, True),
]

for o in orders:
    price, isTakeout, isMember, hasSpecialPromotion = o
    print("Total price: ", calculate_final__price(price, isTakeout, isMember


# Calculating with default params
print("\nusing default params, \nTotal price: ",calculate_final__price(60, T
```

```
Total price:  214.2
Total price:  149.94
Total price:  129.21615
Total price:  239.26139999999998

using default params,
Total price:  63.0
```

Question 7: Write a Write a Python program using map/reduce/filter and lambda.

- To find palindromes in a given list of strings
- Given a list of strings, remove all strings having first character as digit
- Given a list of numbers, find maximum in the list
- Given a list of tuples containing two integers, remove all tuples where second element in tuple is not a factor of first element.
- To find intersection of two given arrays

In [12]:
```
# palindromes
words = ["madam", "riaz", "holo", "123g321", "golang", "malayalam"]
print("palindromes:")
print(list(filter(lambda s: s == s[::-1], words)))

# Given a list of strings, remove all strings having first character as digi
no_digits = list(filter(lambda s: not s[0].isdigit(), words))
print("\n\nremove all strings having first character as digit: ")
print(no_digits)

# Given a list of numbers, find maximum in the list
from functools import reduce
numbers = [45, 22, 89, 12, 77]
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print("\n\n find maximum in the list:")
print(maximum)

# Given a list of tuples containing two integers, remove all tuples where se
t = [(10, 2), (15, 4), (20, 5), (8, 3)]
f = list(filter(lambda t: t[0] % t[1] == 0, t))
print("\n\nremove all tuples where second element in tuple:")
print(f)

# To find intersection of two given arrays
list_a = [1, 2, 3, 4, 5]
list_b = [3, 4, 5, 6, 7]
intersection = list(filter(lambda x: x in list_b, list_a))
print("\n\nTo find intersection of two given arrays:")
print(intersection)
```

```
palindromes:
['madam', '123g321', 'malayalam']


remove all strings having first character as digit:
['madam', 'riaz', 'holo', 'golang', 'malayalam']


 find maximum in the list:
89


remove all tuples where second element in tuple:
[(10, 2), (20, 5)]


To find intersection of two given arrays:
[3, 4, 5]
```

Question 8: Given a dict where values are not unique, write a function to create a new dict where the key is the value and the value is concatenated keys of the original dict and return it.

Input: {'apple':'fruit', 'cat':'mammal','beans':'veg','dog':'mammal', 'mango':'fruit'}
Output: {'fruit':['apple','mango'],'mammal':['cat','dog'],'veg': ['beans']}

In [13]:
```
i = {'apple':'fruit', 'cat':'mammal','beans':'veg','dog':'mammal', 'mango':'

op = {}
```

```
for k,v in i.items():
    # print("k: ",k )
    # print("v: ",v )
    # op[v] = [].append(k)
    if op.get(v)==None:
        op[v] = [k]
    else:
        op[v].append(k)
print(op)
```

```
{'fruit': ['apple', 'mango'], 'mammal': ['cat', 'dog'], 'veg': ['beans']}
```

Question 9:
Weather station records temperature for 7 days (morning & evening). Create a 7×2 array.

Find:

- Daily average temperature
- Maximum temperature recorded
- Extract only evening temperatures.

NOTE: Use numpy to solve the problem.

In [14]:
```
import numpy as np
import random as r

r.seed(0)
# generate array
recording = np.ones((7,2))
print(recording)

# populate with value
for i in range(len(recording)):
    recording[i][0] = r.randrange(20,30)
    recording[i][1] = r.randrange(20,30)
print("\n populate array with value")
print(recording)

# for i in recording:
#     np.average(i)
#     print("daily average of ",i[0]," and ", i[1], " is : ", np.average(i))

# better way
print("\navg: ", np.mean(recording, axis=1))

print("\nonly eve temp:")
recording[:, 1]
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]

 populate array with value
[[26. 26.]
 [20. 24.]
 [28. 27.]
 [26. 24.]
 [27. 25.]
 [29. 23.]
 [28. 22.]]

avg:  [26.  22.  27.5 25.  26.  26.  25. ]

only eve temp:
```

Out[14]:
```
array([26., 24., 27., 24., 25., 23., 22.])
```

Question 10:

Consider a employee.txt: (Use pandas)

```
csv
EmpID,Name,Dept,Salary,Experience
101,ABC,HR,35000,10
102,DEF,IT,60000,5
103,AAA,Finance,45000,3
104,CDE,IT,70000,7
105,BCB,HR,40000,4
106,ADB,Finance,50000,6
107,BDA,IT,65000,5
108,ADC,HR,38000,3
```

Write a code for the following.

- Display first 5 employees
- Display only Name column
- Display Name and Salary
- Find average salary
- Find highest salary
- Find employee with highest salary
- Display IT department employees
- Display employees with salary > 50000
- Find average salary department-wise
- Find most experienced employee
- Find highest paid employee in each department
- Add column "Bonus"= 10% of Salary

In [15]:
```
import pandas as pd

df = pd.read_csv("/Users/apple/Documents/PES/github/python/assignments/Manda

print("\nDisplay first 5 employees")
print(df.head(5))

print("\nDisplay only Name column")
print(df["Name"])

print("\nDisplay Name and Salary")
print(df[["Name", "Salary"]])
```

```
Display first 5 employees
   EmpID Name     Dept  Salary  Experience
0    101  ABC       HR   35000          10
1    102  DEF       IT   60000           5
2    103  AAA  Finance   45000           3
3    104  CDE       IT   70000           7
4    105  BCB       HR   40000           4

Display only Name column
0    ABC
1    DEF
2    AAA
3    CDE
4    BCB
5    ADB
6    BDA
7    ADC
Name: Name, dtype: object

Display Name and Salary
  Name  Salary
0  ABC   35000
1  DEF   60000
2  AAA   45000
3  CDE   70000
4  BCB   40000
5  ADB   50000
6  BDA   65000
7  ADC   38000
```

In [16]:
```
print("\nFind average salary")
print(df["Salary"].mean())

print("\nFind highest salary")
print(df["Salary"].max())

print("\nFind employee with highest salary")
print(df.loc[df["Salary"].idxmax()])
```

```
Find average salary
50375.0

Find highest salary
70000

Find employee with highest salary
EmpID           104
Name            CDE
Dept             IT
Salary        70000
Experience        7
Name: 3, dtype: object
```

In [17]:
```
print("\nDisplay IT department employees")
print(df[df['Dept'] == 'IT'])

print("\nDisplay employees with salary > 50000")
print(df[df['Salary'] > 50000])

print("\nFind average salary department-wise")
print(df.groupby('Dept')['Salary'].mean())
```

```
Display IT department employees
   EmpID Name Dept  Salary  Experience
1    102  DEF   IT   60000           5
3    104  CDE   IT   70000           7
6    107  BDA   IT   65000           5

Display employees with salary > 50000
   EmpID Name Dept  Salary  Experience
1    102  DEF   IT   60000           5
3    104  CDE   IT   70000           7
6    107  BDA   IT   65000           5

Find average salary department-wise
Dept
Finance    47500.000000
HR         37666.666667
IT         65000.000000
Name: Salary, dtype: float64
```

In [18]:
```
## print("\nFind most experienced employee")
print(df.loc[df['Experience'].idxmax()])

print("\nFind highest paid employee in each department")
print(df.loc[df.groupby('Dept')['Salary'].idxmax()])

print("\nAdd column 'Bonus'= 10% of Salary")
df['Bonus'] = df['Salary'] * 0.10
print(df)
```

```
EmpID           101
Name            ABC
Dept             HR
Salary        35000
Experience       10
Name: 0, dtype: object

Find highest paid employee in each department
   EmpID Name     Dept  Salary  Experience
5    106  ADB  Finance   50000           6
4    105  BCB       HR   40000           4
3    104  CDE       IT   70000           7

Add column 'Bonus'= 10% of Salary
   EmpID Name     Dept  Salary  Experience   Bonus
0    101  ABC       HR   35000          10  3500.0
1    102  DEF       IT   60000           5  6000.0
2    103  AAA  Finance   45000           3  4500.0
3    104  CDE       IT   70000           7  7000.0
4    105  BCB       HR   40000           4  4000.0
5    106  ADB  Finance   50000           6  5000.0
6    107  BDA       IT   65000           5  6500.0
7    108  ADC       HR   38000           3  3800.0
```

In [ ]:
```

```